MongoDB CRUD Practical using node.js

Requirements

- **Node.js** installed
- **MongoDB** installed and running

Setup

1. Create project folder and open terminal inside it

2. Initialize project

```
npm init -y
```

3. Install MongoDB driver

```
npm install mongodb
```

4. Create a file named `index.js`

How to Run

```
node index.js
```

Steps to Perform Each Operation

Step 1 — Connect & Create Collection

This runs automatically when you start. It connects to MongoDB, creates a `Student` database and a `users` collection.

=> index.js:

```
const { MongoClient } = require('mongodb');

// Connection URL - Copy from MongoDB Compass and paste the URL below
const url = 'mongodb://localhost:27017';
const client = new MongoClient(url);

async function connectDB() {
    try {

        // Use connect method to connect to the server
        await client.connect();
        console.log('Connected to MongoDB...!');

        // Select database
        const db = client.db('Student_MCA');  // Creates 'Student' DB

        // Create collection
        await db.createCollection('users');
        console.log('Collection created...!');

        const collection = db.collection('users');

        /// Here, perform Insert, Update, and Delete operations, as well as show records and other basic

    } catch (err) {
            console.error('Error:', err);
    } finally {
            await client.close();  // Always close connection
    }
}
connectDB();
```

Expected output:

```
Connected to MongoDB!
Collection created!
```

Step 2 — Insert ONE Record

```
const result = await collection.insertOne({
    name: 'Meet Patel',
    age: 22,
    city: 'Surat',
    status: 'active'
});
console.log('Inserted ID:', result.insertedId);
```

Expected output:

```
Inserted ID: 64f1a2b3c4d5e6f7a8b9c0d1
```

Step 3 — Insert MANY Records

```
const users = [
    { name: 'Raj Shah', age: 21, city: 'Ahmedabad', status: 'active' },
    { name: 'Priya Mehta', age: 23, city: 'Vadodara', status: 'inactive' },
    { name: 'Amit Joshi', age: 25, city: 'Surat', status: 'active' }
];
const manyResult = await collection.insertMany(users);
console.log('Inserted Count:', manyResult.insertedCount);
```

Expected output:

```
Inserted Count: 3
```

Step 4 — Find ONE Document

```
const user = await collection.findOne({ name: 'Raj Shah' });
if (user) {
    console.log('Found:', user);
} else {
    console.log('User not found.');
}
```

Expected output:

```
Found: { _id: ..., name: 'Raj Shah', age: 21, city: 'Ahmedabad', status: 'active' }
```

Step 5 — Find ALL Documents

```
const users = await collection.find().toArray();
console.log('All users:', users);
```

Step 6 — Filter Documents

Active users only:

```
const active = await collection.find({ status: 'active' }).toArray();
console.log('Active users:', active);
```

Users aged above 20:

```
const results = await collection.find({ age: { $gt: 20 } }).toArray();
console.log('Users aged:', results);
```

Step 7 — Update ONE Record

```
const r1 = await collection.updateOne(
    { name: 'Priya Mehta' },
    { $set: { city: 'Ahmedabad' } }
);
console.log(r1.matchedCount, r1.modifiedCount);
```

Expected output:

```
1 1
```

Step 8 — Update MANY Records

```
const r2 = await collection.updateMany(
    { city: 'Surat' },
    { $set: { status: 'active' } }
);
console.log(`${r2.modifiedCount} records updated`);
```

Step 9 — Delete ONE Record

```
const r1 = await collection.deleteOne({ name: 'Meet Patel' });
console.log('Deleted:', r1.deletedCount);
```

Step 10 — Delete MANY Records

```
const r2 = await collection.deleteMany({ status: 'inactive' });
console.log(`${r2.deletedCount} docs deleted`);
```

Quick Reference

Operation	Method
Insert one	`collection.insertOne({})`
Insert many	`collection.insertMany([])`
Find one	`collection.findOne({})`
Find all	`collection.find().toArray()`
Update one	`collection.updateOne({}, {})`
Update many	`collection.updateMany({}, {})`
Delete one	`collection.deleteOne({})`
Delete many	`collection.deleteMany({})`

> Submit here - https://forms.gle/3bBXEKQyKCBF2d5C8